

LOG430 – Laboratoire 00

Nom : Amjad Lekhdar **Date :** 13 mai 2026

Lien GitHub :

<https://github.com/Amjad-Lekhdar/LOG430-Labo00>

Question 1

Énoncé

Expliquez ce que Pytest affiche lorsqu'un test échoue et donnez un exemple avec un test volontairement incorrect.

Réponse

Pytest affiche sur le terminal quel test a échoué, le fichier concerné, la ligne exacte ainsi que la différence entre la valeur attendue et la valeur obtenue. Par exemple si j'ajoute de façon volontaire ce test:

```
def test_addition_erreur():  
    my_calculator = Calculator()  
    assert my_calculator.addition(2, 3) == 6
```

Sortie obtenue du terminal:

```
FAILED src/tests/test_calculator.py::test_addition_erreur - assert 5 == 6
```

En gros, le test a échoué car `addition(2,3)` retourne 5, alors qu'il s'attendait 6.

Question 2

Énoncé

Expliquez le rôle des étapes `checkout` et `setup` dans le pipeline GitHub Actions de votre projet. Incluez un exemple de sortie terminal.

Réponse

Dans le pipeline CI (Continuous Integration), l'étape `checkout` a pour rôle de télécharger le code du dépôt Github dans l'environnement temporaire du runner. En l'absence de cette étape, GitHub Actions ne pourrait pas accéder aux fichiers du projet. Dans mon fichier `ci.yml`, cette étape utilise `actions/checkout@v3`. Par la suite, l'étape `setup` s'occupe des installations de Python 3.11 avec `actions/setup-python@v4` et les dépendances avec `pip`, et s'occupe de l'exécution des tests avec `python -m pytest`.

Question 3

Énoncé

Expliquez à quoi sert la commande **top** sous Linux et quelles informations importantes elle permet d'observer sur un serveur.

Réponse

La commande **top** sert à surveiller l'état du serveur en temps réel. Elle assure l'affichage de l'utilisation du processeur, la mémoire RAM, le swap, le nombre de tâches/processus, le temps depuis le démarrage du système et les processus les plus actifs.

Exemple de sortie :

```
top - 21:30:12 up 1 day,  2:15,  1 user,  load average: 0.05, 0.03, 0.01
Tasks: 112 total,   1 running, 111 sleeping
%Cpu(s):  2.0 us,   1.0 sy, 97.0 id
MiB Mem :  3920.0 total,  1200.0 free,   900.0 used,  1820.0 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used

PID USER  PR  NI    VIRT    RES  %CPU  %MEM COMMAND
123 root   20   0  123456  54320   1.2   1.4 python
456 root   20   0  987654  80000   0.5   2.0 docker
```

Exemples d'informations utiles : voir si Docker consomme trop de RAM, si un processus Python bloque le CPU, ou si la VM manque de mémoire.
